

ELEC210K Computer Programming

Short Coding Practice

Lecture 6: Procedural Programming & Functions

Question 1: Value-Returning Function Implementation

Complete the body of the function `findMax` which takes three integers by value as parameters, determines the largest value among them, and returns it to the caller.

```
#include <iostream>
using namespace std;

// Function declaration and definition
int findMax(int num1, int num2, int num3) {

    // TODO: WRITE THE CONTENT OF THE FUNCTION HERE

}
```

Question 2: Declaring Function Prototypes (Forward Declaration)

Write the correct function prototype (forward declaration) above `main()` for the `computePower` function so that the code compiles successfully without moving the function definition block.

```
#include <iostream>
using namespace std;

// TODO: DECLARE THE FUNCTION PROTOTYPE HERE

int main() {
    double result = computePower(2.5, 3);
    cout << "Result: " << result << endl;
}
```

```

    return 0;
}

// Function definition is placed below main
double computePower(double base, int exponent) {
    double res = 1.0;
    for (int i = 0; i < exponent; i++) {
        res *= base;
    }
    return res;
}

```

Question 3: Void Function with Side Effects (Console Output)

Complete the content of the `printPattern` function. It takes an integer parameter `n` and must output a sequence of asterisks `\*` of length `n` to the standard console followed by a newline. The function returns no value.

```

#include <iostream>
using namespace std;

// Void function that returns nothing
void printPattern(int n) {

    // TODO: WRITE YOUR CODE HERE TO PRINT THE PATTERN

}

```

Question 4: Pass-by-Value Isolation and Constraints

Predict what the following program will output to the standard console. This checks your comprehension of parameter scope separation under C++ pass-by-value data isolation rules.

```

#include <iostream>
using namespace std;

void modifyValue(int x) {
    x = x + 10;
    cout << "Inside function: " << x << endl;
}

```

```
int main() {  
    int num = 5;  
    modifyValue(num);  
    cout << "Inside main: " << num << endl;  
    return 0;  
}
```

```
// TODO: WRITE THE CONSOLE OUTPUT BELOW
```

```
//  
// Line 1:  
// Line 2:
```

Question 5: Functional Decomposition and Math Library Calls

Complete the code for `calculateHypotenuse` to find the hypotenuse of a right-angled triangle using the standard math components `sqrt()` and `pow()`. Make sure to call the correct syntax from the included `` library header.

```
#include <iostream>  
#include <cmath> // For mathematical library utilities  
using namespace std;  
  
double calculateHypotenuse(double sideA, double sideB) {  
  
    // TODO: WRITE YOUR RETURN STATEMENT HERE
```

```
}
```

Lecture 7: Introduction to Classes & Objects

Question 1: Data Member Encapsulation and Access Modifiers

Complete the skeleton of the `Circle` class by organizing attributes and methods into their proper access scopes. The attributes (`radius`) must be protected from external modification, while the constructor must be accessible from outside the class.

```
#include <iostream>
using namespace std;

class Circle {
    // TODO: CHOOSE AND SPECIFY ACCESS MODIFIERS HERE
    double radius;

    Circle(double r) {
        radius = r;
    }
};
```

Question 2: Parameterized Constructor and Initializer Lists

Complete the implementation of the parameterized constructor for the `Time` class using a member initializer list to initialize `hours`, `minutes`, and `seconds` directly.

```
#include <iostream>
using namespace std;

class Time {
private:
    int hours;
    int minutes;
    int seconds;
public:
    // TODO: Complete the constructor header using an initializer list
    Time(int h, int m, int s)
```

```
// WRITE YOUR INITIALIZER LIST HERE
```

```
{  
    // Body left empty  
}  
};
```

Question 3: Implementing Accessors (Getter Functions)

Complete the body of the accessor function `getRadius()` which safely returns the private data member `radius` to external callers without allowing modification.

```
#include <iostream>  
using namespace std;  
  
class Circle {  
private:  
    double radius;  
public:  
    Circle(double r) : radius(r) {}  
  
    // Accessor function (Getter)  
    double getRadius() const {  
  
        // TODO: WRITE YOUR CODE HERE
```

```
    }  
};
```

Question 4: Implementing Mutators (Setter Functions with Validation)

Complete the body of the mutator function `setRadius()`. It should validate the incoming parameter; if `r` is greater than 0, update the private `radius` member variable. Otherwise, leave it unchanged.

```
#include <iostream>  
using namespace std;
```

```

class Circle {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}

    // Mutator function (Setter)
    void setRadius(double r) {

        // TODO: WRITE YOUR CODE HERE WITH VALIDATION

    }
};

```

Question 5: Instantiating Objects and Calling Member Behaviors

Inside the `main()` function, write code to instantiate an object of the `Time` class named `lunchTime` with values (12, 30, 0) and then call its `print()` method.

```

#include <iostream>
using namespace std;

class Time {
private:
    int hours, minutes, seconds;
public:
    Time(int h, int m, int s) : hours(h), minutes(m), seconds(s) {}
    void print() const {
        cout << hours << ":" << minutes << ":" << seconds << endl;
    }
};

int main() {

    // TODO: INSTANTIATE LUNCHTIME OBJECT AND CALL PRINT

}

return 0;
}

```

Lecture 9 Quiz: References & Parameter Passing

Question 1: Declaring and Binding a Reference Variable

Write the missing code to declare a reference variable named `rScore` and immediately bind/alias it to the existing integer variable `score`.

```
#include <iostream>
using namespace std;

int main() {
    int score = 92;

    // TODO: DECLARE rScore AND BIND IT TO score

    rScore = 95; // Changes 'score' to 95 through the reference alias
    cout << "Score: " << score << endl;
    return 0;
}
```

Question 2: Implementing a Swap Function (Pass-by-Reference)

Complete the body of the `swapValues` function using references. The function must accept two integer values by reference, swapping their underlying variables directly without copying.

```
#include <iostream>
using namespace std;

// Function using reference parameters to swap actual variables
void swapValues(int& first, int& second) {

    // TODO: WRITE THE SWAP ALGORITHM HERE

}

int main() {
    int x = 10, y = 20;
    swapValues(x, y); // x becomes 20, y becomes 10
}
```

```
    return 0;
}
```

Question 3: Constant References (Pass-by-Const-Reference)

Complete the function header and content below to pass a large `string` object efficiently by constant reference (`const string&`). This eliminates copying while protecting the original object from being accidentally modified.

```
#include <iostream>
#include <string>
using namespace std;

// TODO: Complete the function header to pass 'str' by constant reference
void printMessage(
)
{
    // TODO: PRINT THE STRING TO COUT HERE

}
}
```

Question 4: Return-by-Reference Mechanism

Complete the function header and implementation for `getLarger` so that it returns a reference to the larger of two integer parameters. This allows the returned value to be used directly on the left-hand side of an assignment expression (lvalue).

```
#include <iostream>
using namespace std;

// TODO: Complete the function header to return an int reference
getLarger(int& a, int& b)
{
    // TODO: RETURN A REFERENCE TO THE LARGER INITIAL INT

}

int main() {
    int x = 50, y = 80;
```



```
getLarger(x, y) = 100; // Overwrites y to 100 directly
return 0;
}
```

Question 5: Side Effects of Side-by-Side Reference Changes

Predict the tracking console outputs for `a`, `b`, and `ref` throughout the following main execution block. This verifies your structural mental model of how modifications to references dynamically interact with base memory addresses.

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int b = 15;
    int& ref = a; // ref binds to a

    ref = b; // What does this do?
    ref = 30;

    cout << "a: " << a << endl;
    cout << "b: " << b << endl;
    return 0;
}

// TODO: WRITE THE EXPECTED OUTPUT BELOW
//
// Value of a:
// Value of b:
```

Lecture 11 Quiz: Inheritance & Class Relationships

Question 1: Public Inheritance Header Syntax

Complete the class definition header for `Student` so that it inherits publicly from the `Person` base class.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string nm) : name(nm) {}
};

// TODO: WRITE THE CLASS HEADER WITH INHERITANCE

{
private:
    double gpa;
public:
    Student(string nm, double g) : Person(nm), gpa(g) {}
};
```

Question 2: Base Class Constructor Initialization

Complete the constructor initialization list of the derived class `Student`. The derived constructor must explicitly pass the parameter `nm` upward to the base class `Person` constructor before initializing its own member variable `gpa`.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string nm) : name(nm) {}
};
```

```

class Student : public Person {
private:
    double gpa;
public:
    // Complete the constructor implementation below
    Student(string nm, double g)

        // TODO: WRITE MEMBER INITIALIZATION LIST HERE

{
    // Body remains empty
}
};

```

Question 3: Member Function Redefinition (Function Overriding)

Complete the `print()` member function inside the derived class `Student`. It should first invoke the base class version of `print()` to display the person's name, and then print the student's unique `gpa` attribute.

```

#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
public:
    Person(string nm) : name(nm) {}
    void print() const { cout << "Name: " << name << endl; }
};

class Student : public Person {
private:
    double gpa;
public:
    Student(string nm, double g) : Person(nm), gpa(g) {}

    void print() const {

```

```
// TODO: WRITE YOUR CODE HERE
```

```
    }  
};
```

Question 4: Protected Member Variable Access

Fill in the blank inside the derived class function to demonstrate that `protected` members of a base class are directly accessible inside member functions of derived classes, unlike `private` members.

```
#include <iostream>  
#include <string>  
using namespace std;  
  
class Person {  
protected:  
    string name; // Accessible to derived classes  
public:  
    Person(string nm) : name(nm) {}  
};  
  
class Student : public Person {  
private:  
    double gpa;  
public:  
    Student(string nm, double g) : Person(nm), gpa(g) {}  
  
    void changeName(string newName) {  
  
        // TODO: ASSIGN newName DIRECTLY TO THE INHERITED PROTECTED MEMBER
```

```
    }  
};
```

Question 5: Order of Constructor & Destructor Execution

Write down what the following program will output to the standard console when an instance of `Derived` is instantiated and immediately destroyed at the end of `main()` scope. This tests your understanding of constructor/destructor execution order.

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base Con" << endl; }
    ~Base() { cout << "Base Des" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived Con" << endl; }
    ~Derived() { cout << "Derived Des" << endl; }
};

int main() {
    Derived obj;
    return 0;
}

// TODO: WRITE THE EXPECTED OUTPUT BELOW
//
// 1.
// 2.
// 3.
// 4.
```

Lecture 12 Quiz: Polymorphism & Abstract Classes

Question 1: Pure Virtual Function Definition (Abstract Base Class)

Declare a pure virtual function named `getArea` that returns a `double` and does not modify the object state (`const`). This makes the `Shape` class an abstract base class.

```
#include <iostream>
using namespace std;

class Shape {
public:

    // TODO: DECLARE THE PURE VIRTUAL FUNCTION 'getArea' HERE

};
```

Question 2: Polymorphic Overriding in Derived Class

Complete the derived class `Rectangle` by overriding the `getArea()` function to compute and return the correct area using its width and height.

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double getArea() const = 0; // Pure virtual function
};

class Rectangle : public Shape {
private:
    double width;
    double height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}

    // Overriding the getArea function
    double getArea() const override {
```

```
// TODO: WRITE YOUR CODE HERE
```

```
    }  
};
```

Question 3: Virtual Destructors for Dynamic Memory Deallocation

Add a virtual destructor to the `Base` class so that when a derived class object is deleted through a base class pointer, both the base and derived destructors are properly invoked, preventing memory leaks.

```
#include <iostream>  
using namespace std;  
  
class Base {  
public:  
    Base() { cout << "Base constructor" << endl; }
```

```
    // TODO: WRITE THE VIRTUAL DESTRUCTOR HERE
```

```
};  
  
class Derived : public Base {  
public:  
    Derived() { cout << "Derived constructor" << endl; }  
    ~Derived() { cout << "Derived destructor (resources freed)" << endl; }  
};
```

Question 4: Runtime Dynamic Binding using Pointers

Complete the body of the global function `printShapeArea` which accepts a pointer to the abstract `Shape` class. The function should invoke the `getArea()` function polymorphically and print it.

```
#include <iostream>  
using namespace std;  
  
class Shape {  
public:  
    virtual double getArea() const = 0;  
};
```

```
// Global standalone function demonstrating dynamic binding
void printShapeArea(const Shape* shapePtr) {

    // TODO: WRITE YOUR CODE HERE

}
```

Question 5: Diamond Inheritance & Virtual Base Classes

Declare classes `Sub1` and `Sub2` using virtual base inheritance from `Person` so that a child class inheriting from both `Sub1` and `Sub2` will only contain a single, shared instance of `Person`.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
    string name;
};

// TODO: DEFINE SUB1 AND SUB2 WITH VIRTUAL BASE INHERITANCE

class TeachingAssistant : public Sub1, public Sub2 {
    // Contains only one single copy of 'name'
};
```


Lecture 13 Quiz: Object Roles & Operator Overloading

Question 1: Constant Member Function (Host Object Protection)

Complete the body of the const member function `print()` so that it prints the fraction in the format `numerator/denominator` to the standard output. The function must not modify the host object.

```
#include <iostream>
using namespace std;

class Fraction {
private:
    int numer;
    int denom;
public:
    Fraction(int n, int d) : numer(n), denom(d) {}

    // Prints the fraction as numer/denom without modifying the host object
    void print() const {

        // TODO: WRITE YOUR CODE HERE

    }
};
```

Question 2: Member Operator Overloading (Pre-Increment)

Complete the body of the pre-increment operator member function. It should add the denominator to the numerator, normalize or update the object, and return a reference to the modified host object to support chaining (e.g., `++(++f)`).

```
#include <iostream>
using namespace std;

class Fraction {
private:
    int numer;
    int denom;
public:
    Fraction(int n, int d) : numer(n), denom(d) {}

    // Overloading Pre-increment Operator (++f)
    Fraction& operator++() {
```

```
// TODO: WRITE YOUR CODE HERE
```

```
    }  
};
```

Question 3: Member Operator Overloading (Post-Increment)

Complete the body of the post-increment operator member function. Note the dummy `int` parameter used to distinguish it from the pre-increment operator. It should save the current state in a temporary object, invoke the pre-increment behavior, and return the unchanged temporary object by value.

```
#include <iostream>  
using namespace std;  
  
class Fraction {  
private:  
    int numer;  
    int denom;  
public:  
    Fraction(int n, int d) : numer(n), denom(d) {}  
  
    Fraction& operator++(); // Assume pre-increment is implemented  
  
    // Overloading Post-increment Operator (f++)  
    const Fraction operator++(int) {  
  
        // TODO: WRITE YOUR CODE HERE
```

```
    }  
};
```

Question 4: Non-Member Friend Operator Overloading (Binary Addition)

Complete the body of the binary addition operator overloaded as a non-member friend function. It takes two parameter objects by constant reference and returns a new Fraction object representing their sum.

```

#include <iostream>
using namespace std;

class Fraction {
private:
    int numer;
    int denom;
public:
    Fraction(int n, int d) : numer(n), denom(d) {}

    friend const Fraction operator+(const Fraction& left, const Fraction& right);
};

// Friend function implementation
const Fraction operator+(const Fraction& left, const Fraction& right) {

    // TODO: WRITE YOUR CODE HERE
}

```

Question 5: Copy Assignment Operator Overloading

Complete the body of the overloaded copy assignment operator (`operator=`). It must check for self-assignment using the `this` pointer, copy the data members, and return a reference to the host object to enable assignment chaining.

```

#include <iostream>
using namespace std;

class Fraction {
private:
    int numer;
    int denom;
public:
    Fraction(int n, int d) : numer(n), denom(d) {}

    // Overloading Copy Assignment Operator
    Fraction& operator=(const Fraction& right) {

        // TODO: WRITE YOUR CODE HERE
}

```

```
}  
};
```